National University of Computer and Emerging Sciences



Lab Manual 06
Artificial Intelligence

Course Instructor	Ms. Bushra Rashid
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 6 3
Objectives: 3
Activities: 3
Activity 1 (Searching Algorithms): 3
Minimax Algorithm 3
Algorithm: 3
Pseudocode: 3
Node Representation: 4
For example: 4
Alpha–Beta Pruning for Minimax Algorithm 4
Alpha–Beta Values 4
Alpha (α) 4
Beta (β) 4
Pruning Condition 4
Pruning Rules 5
At MAX Node 5
At MIN Node 5
Minimax with Alpha–Beta Pruning 5
Algorithm 5
Input: 5
Output: 5
Steps 5
Pseudocode 6
Activity 2 (Problem Statements / Tasks): 6
1. Minimax Tree Evaluation 6
2. Minimax with Alpha–Beta Pruning 7
3. Alpha–Beta Pruning Efficiency Experiment 7

Lab Manual 6

Objectives:

- ✓ Searching Algorithms
  - Minimax Algorithm
  - Alpha–Beta Pruning
- ✓ Game Playing using Minimax Algorithm with Alpha–Beta Pruning

Activities:

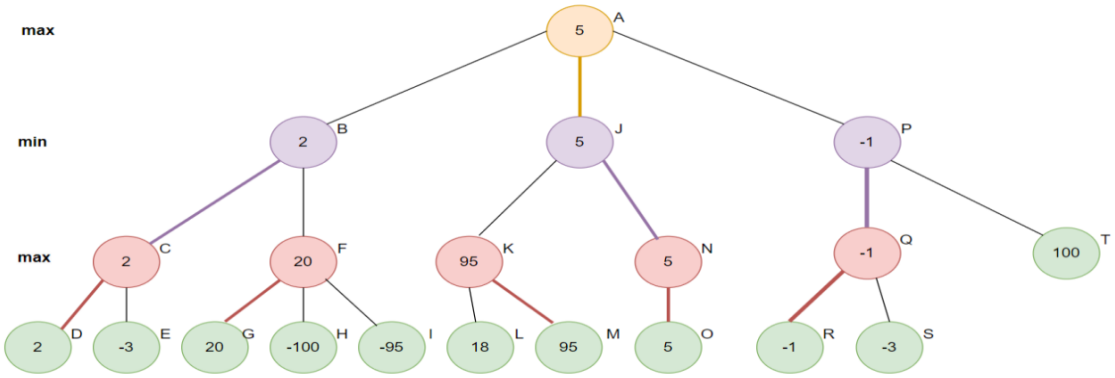
Activity 1 (Searching Algorithms):

Minimax Algorithm

The Minimax algorithm is a recursive decision-making algorithm used for optimal play in two-player games.

It assumes:

- Both players play optimally
- MAX tries to maximize the utility
- MIN tries to minimize the utility



Algorithm:

- Start from the root node
- If the node is a terminal state:
  - Return its utility value
- If it is MAX's turn:
  - Evaluate all child nodes
  - Return the maximum value
- If it is MIN's turn:
  - Evaluate all child nodes
  - Return the minimum value

Pseudocode:

```
MINIMAX(node, isMax):
    if node is terminal:
        return utility(node)
```

```
    if isMax:
        best = -∞
        for each child:
            value = MINIMAX(child, False)
            best = max(best, value)
        return best

    else:
        best = +∞
        for each child:
            value = MINIMAX(child, True)
            best = min(best, value)
        return best
```

Node Representation:

A node in a game tree may contain:

- Game state
- List of possible moves
- Player type (MAX or MIN)
- Utility value (for terminal nodes)

For example:

```
class GameNode:
    def __init__(self, value=None, children=None):
        self.value = value
        self.children = children
```

Alpha–Beta Pruning for Minimax Algorithm

The basic Minimax algorithm explores all nodes of the game tree. For games with large branching factors, this becomes computationally expensive.

Alpha–Beta Pruning improves Minimax by:

- Avoiding exploration of branches that cannot affect the final decision
- Reducing the number of nodes evaluated
- Producing the same result as Minimax but faster

Alpha–Beta Values

Two parameters are maintained during search.

Alpha (α)
Best value MAX can guarantee so far.
Initial value: $\alpha = -\infty$

Beta (β)
Best value MIN can guarantee so far.
Initial value: $\beta = +\infty$

Pruning Condition

If at any time:

$$\alpha \geq \beta$$

Remaining nodes in that branch do not need to be explored.

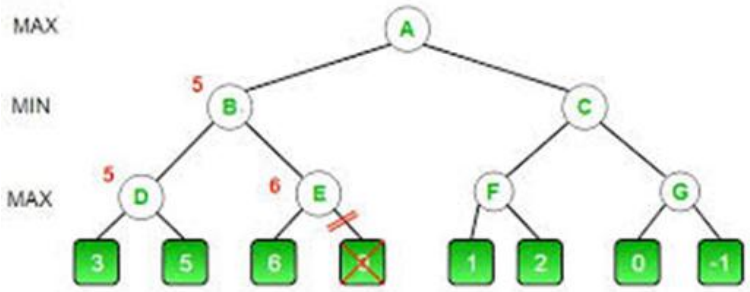
Pruning Rules

At MAX Node

If $current_value \geq \beta$, stop exploring further children. This is called Beta Cut-off.

At MIN Node

If $current_value \leq \alpha$, stop exploring further children. This is called Alpha Cut-off.



Minimax with Alpha–Beta Pruning

Algorithm

Input:

- A game-tree node node
- Current depth depth
- Alpha value α
- Beta value β
- Boolean isMax, indicating whether the current player is the Maximizing player

Output:

- The optimal minimax value of the given node

Steps

- Start at the root node with:
  - α = -∞
  - β = +∞
  - isMax = TRUE
- If the current node is a leaf node, return its utility/value.
  - If the current player is the Maximizing player:
  - Set best = -∞.
  - Visit each child of the current node.
  - Recursively calculate the minimax value of the child assuming the next player is Minimizing.
  - Update best with the maximum value found.
  - Update α with the maximum of the current α and best.
  - If β ≤ α, stop evaluating the remaining children because they cannot affect the final decision. This is Alpha–Beta pruning.
  - Return best.
- If the current player is the Minimizing player:
  - Set best = +∞.
  - Visit each child of the current node.

- Recursively calculate the minimax value of the child assuming the next player is Maximizing.
- Update best with the minimum value found.
- Update β with the minimum of the current β and best.
- If β ≤ α, stop evaluating the remaining children because they cannot affect the final decision.
- Return best.

- The value returned for the root node represents the optimal value for the Maximizing player.

Pseudocode

```
MINIMAX_ALPHA_BETA(node, isMax, alpha, beta):
    if node is terminal:
        return utility(node)

    if isMax:
        best = -∞

        for each child:
            value = MINIMAX_ALPHA_BETA(child, False, alpha, beta)

            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break        // Prune remaining branches
        return best

    else:
        best = +∞

        for each child:

            value = MINIMAX_ALPHA_BETA(child, True, alpha, beta)

            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break        // Prune remaining branches

        return best
```

Activity 2 (Problem Statements / Tasks):

1. *Minimax Tree Evaluation*

Construct the following game tree manually using GameNode class:

- Root → MAX
- Level 1 → MIN nodes
- Leaves contain values:

- 3, 12
- 8, 2
- 4, 6

Requirements

- Create the tree structure.
- Apply Minimax algorithm.
- Display:
  - Value chosen at each MIN node
  - Final optimal value at root
- Explain:
  - Why MAX chose that value?

2. *Minimax with Alpha–Beta Pruning*

Construct the following game tree using the GameNode class.

Requirements

- Construct the above game tree using the GameNode class.
- Implement the Minimax algorithm.
- Implement Minimax with Alpha–Beta Pruning.
- Display:
  - Optimal value at the root node
  - Value returned by each MIN node
  - Value returned by each MAX node
- Display which branches were pruned during Alpha–Beta pruning.

Compare

Count and display:

- Number of leaf nodes evaluated using Basic Minimax
- Number of leaf nodes evaluated using Alpha–Beta Pruning

Explain:

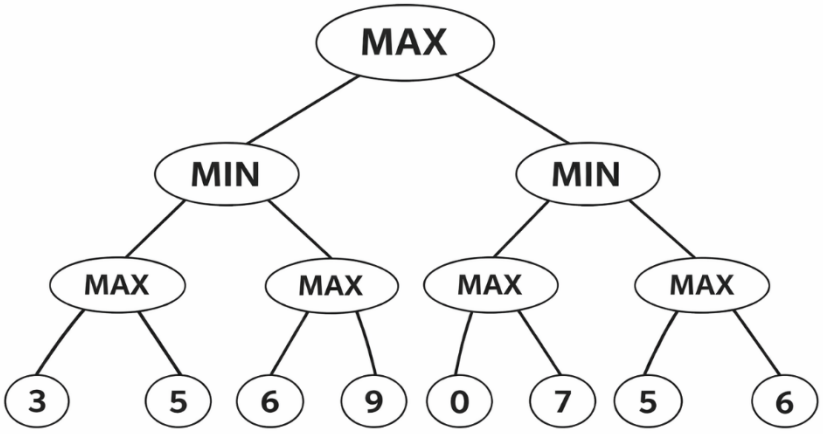
- Why Alpha–Beta Pruning explores fewer nodes.

3. *Alpha–Beta Pruning Efficiency Experiment*

Using the same game tree from Problem 2, analyse the efficiency of Alpha–Beta pruning.

Requirements

- Implement Basic Minimax.
- Implement Minimax with Alpha–Beta Pruning.
- Modify the program to count the number of nodes visited.



Display:

- Nodes visited using Basic Minimax
- Nodes visited using Alpha–Beta Pruning
- Nodes saved due to pruning

Questions

Students must explain:

- Which algorithm evaluated fewer nodes?
- Why does Alpha–Beta Pruning improve performance?
- Does Alpha–Beta change the final optimal value?